FACULTAD DE CIENCIAS E INGENIERÍA

CARRERA DE INGENIERÍA BIOMÉDICA



Entregable 7

Curso:
Fundamentos de Biodiseño

Integrantes:
Medina Pinedo Gabriel Miguelangel
Mendoza Perez Adrian Elioth
Periche Eche Flor Daniela
Pisconte Intor Yehoshua Zahir
Retuerto Rojas Eddy Moises
Riveros Huancahuari Asiry Fiorella

Carrera:
Ing. Biomédica

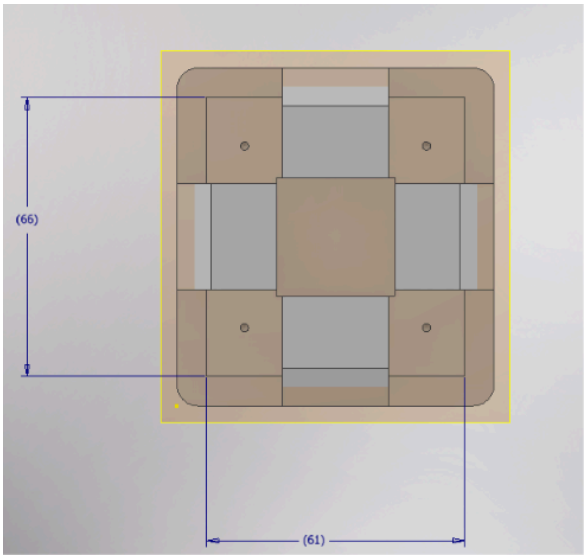
Fecha:
Noviembre 2026

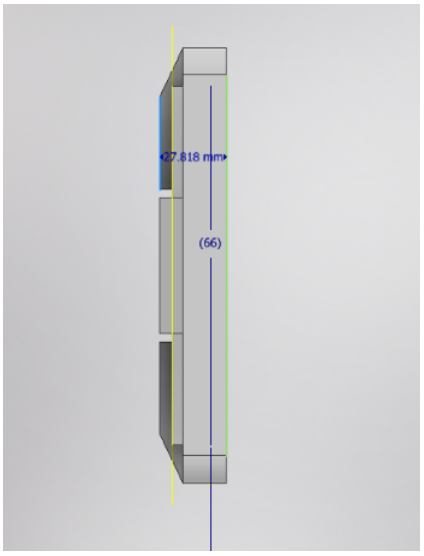
Índice

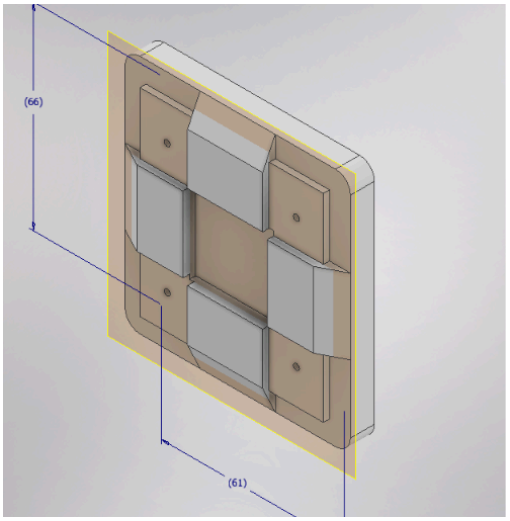
Índice..... 2
1. Implementación preliminar de los subsistemas..... 3
1.1. Mecánica y fabricación..... 3
1.2. Electrónica y sensores..... 3
1.3. Programación y control..... 5
2. Verificación preliminar del diseño por subsistemas..... 6
2.1. Mecánica y fabricación..... 6
2.2. Electrónica y sensores..... 6
2.3. Programación y control..... 6

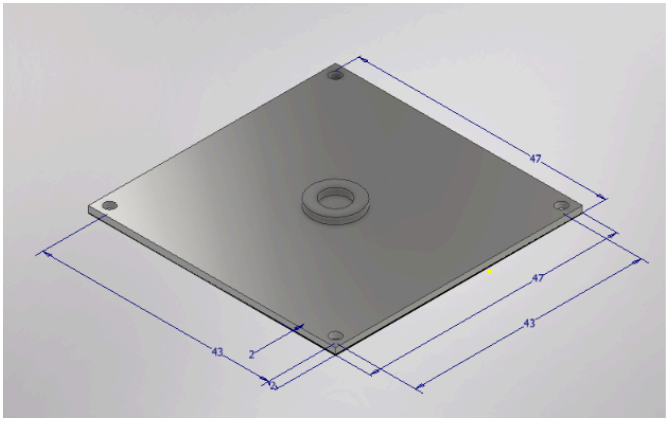
1. Implementación preliminar de los subsistemas

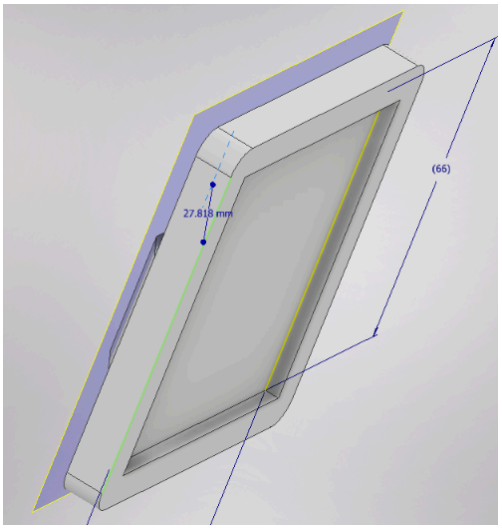
1.1. Mecánica y fabricación

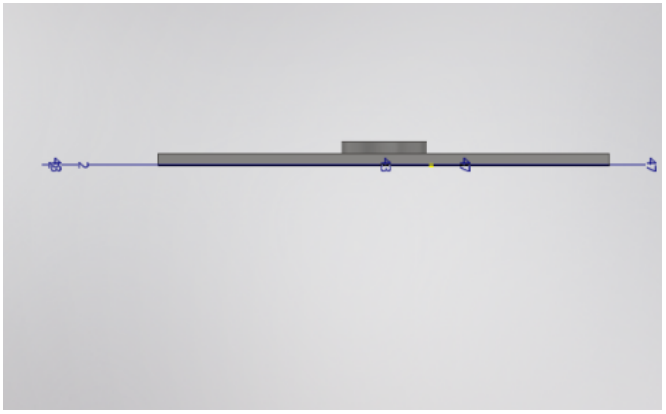


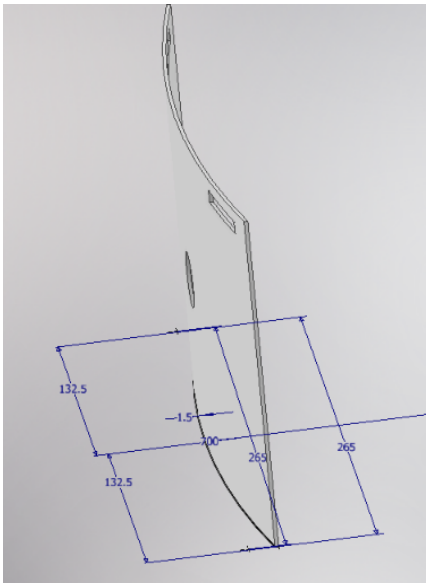


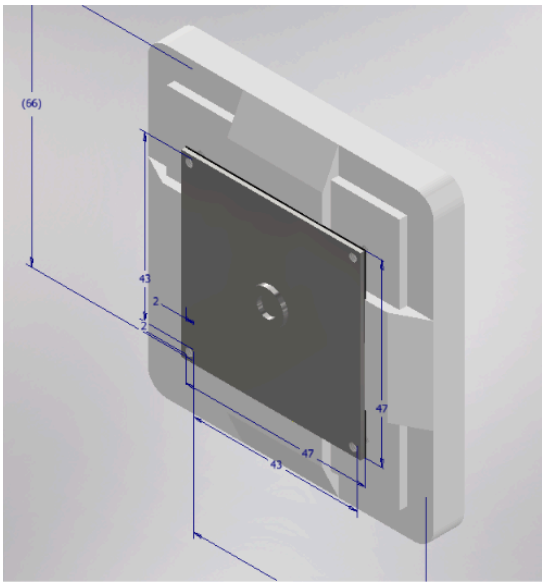


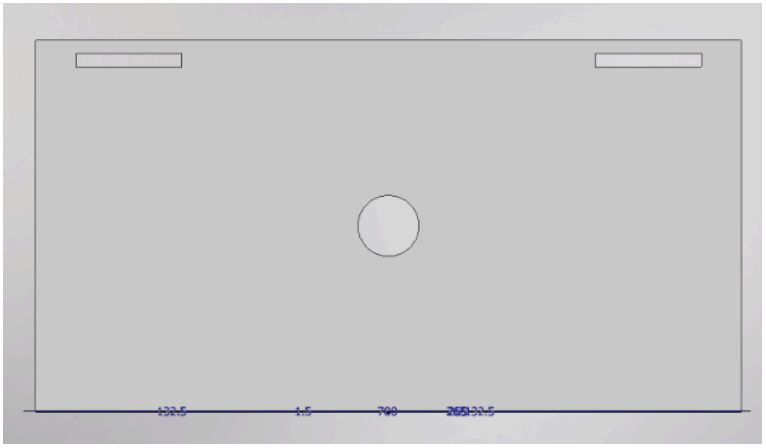


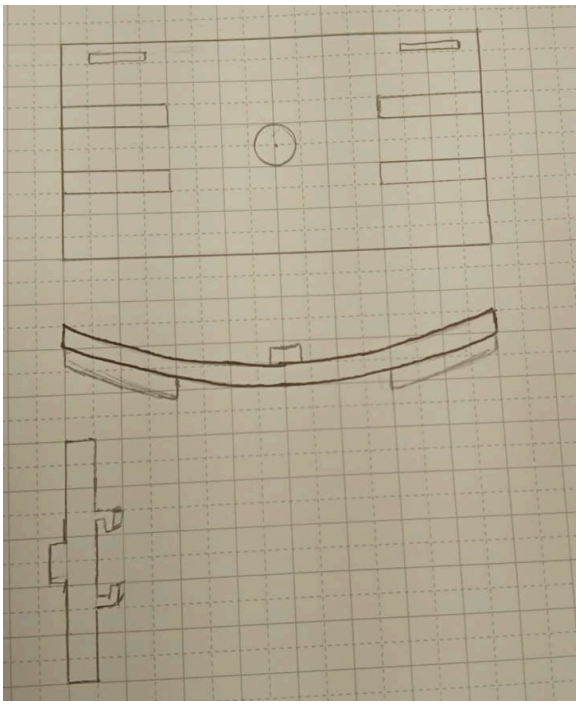





132.5
1.5
265
132.5

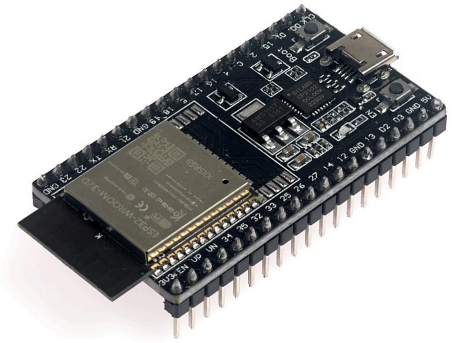

(66)
43
2
47
47
43





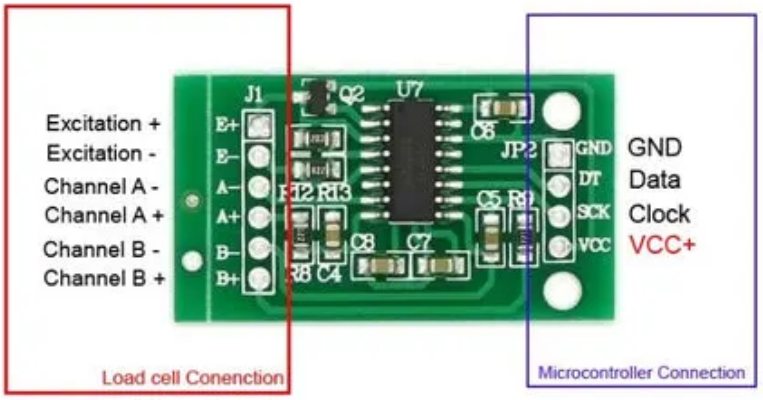
1.2. Electrónica y sensores

- **ESP32 (DOIT ESP32 DEVKIT V1)**



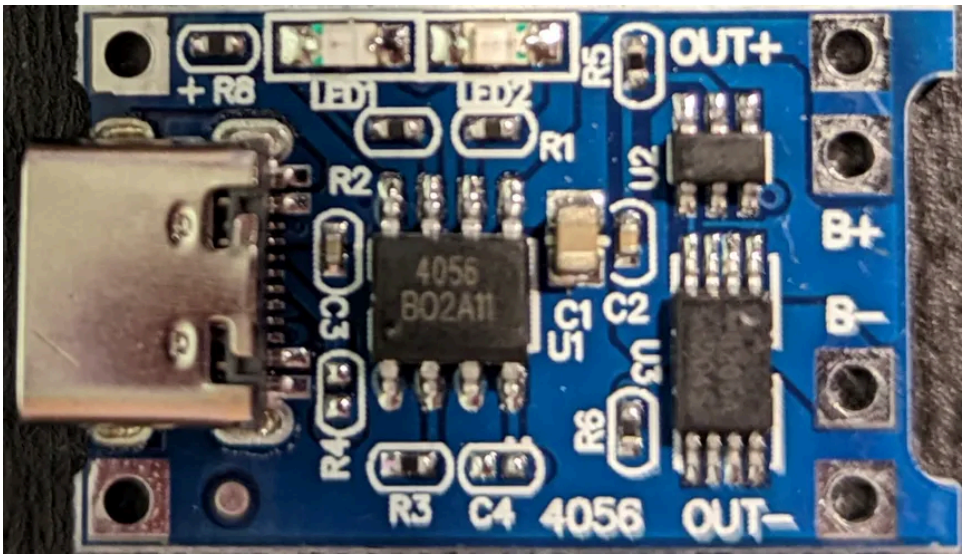
- **Componente:** Microcontrolador ESP32 (DOIT ESP32 DEVKIT V1).
- **Función:** Es la unidad central de procesamiento del prototipo, encargada de recibir y procesar las señales provenientes de los sensores, ejecutar la lógica de control del sistema y gestionar la comunicación inalámbrica mediante Bluetooth y Wi-Fi, permitiendo la interacción y transmisión de datos entre los componentes del dispositivo.

- **XH711 module**



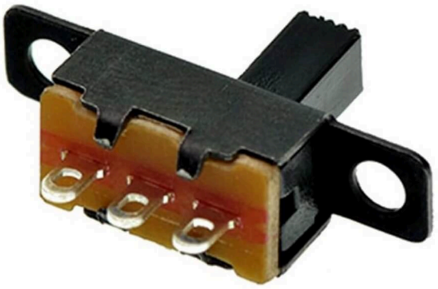
Módulo HX711: Actúa como acondicionador de señal y conversor analógico-digital (ADC) de 24 bits. Se encarga de amplificar la pequeña señal generada por la celda de carga y transmitir la lectura digitalizada al microcontrolador.

- **tp4056 type c**



- **Componente:** Módulo de carga de batería LiPo TP4056 con puerto USB-C.
- **Función:** Administra la carga de la batería del dispositivo y regula la alimentación hacia el microcontrolador y los demás módulos.

- **Interruptor deslizante (SW1).**



- **Componente:** Interruptor deslizante (SW1).
- **Función:** Permite controlar manualmente el encendido y apagado del circuito electrónico, interrumpiendo o habilitando el paso de corriente según su conexión.

- **Conector JST-PH de 2 pines.**



- **Componente:** Conector JST-PH de 2 pines.
- **Función:** Permite establecer una conexión eléctrica entre la batería y el circuito de alimentación del prototipo.

- **Conector PH de 4 pines.**



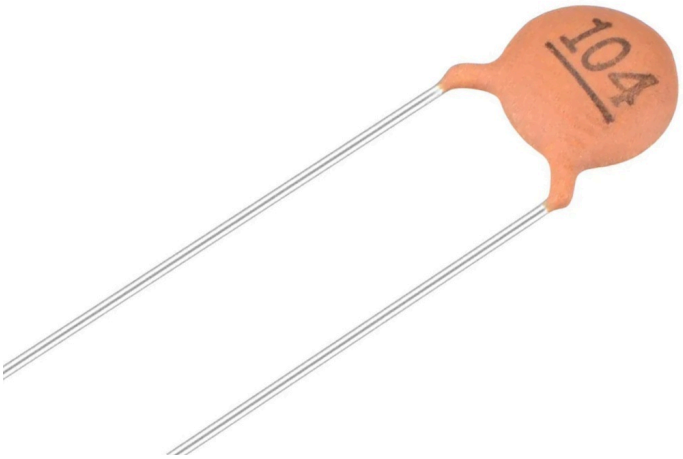
- **Componente:** Conector PH de 4 pines.
- **Función:** Permite realizar conexiones eléctricas entre los componentes del sistema, facilitando la distribución de señales y alimentación.

- **Resistencia de 100 Ω.**



- **Componente:** Resistencia de 100 Ω.
- **Función:** Limita el paso de corriente eléctrica en el circuito, contribuyendo a proteger los componentes electrónicos conectados.

- **Condensador de 100nF.**



- **Componente:** Condensador de 100nF.
- **Función:** Ayuda a reducir las variaciones y perturbaciones de voltaje en el circuito, contribuyendo a estabilizar la alimentación eléctrica.

1.3. Programación y control

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "HX711.h"
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>

// ========================================
// CONFIGURACIÓN DE PANTALLA OLED
// ========================================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ========================================
// PINES Y HARDWARE (ESP32)
// ========================================
const int HX711_DOUT_PIN = 4;  // Pin GPIO 4 del ESP32
const int HX711_SCK_PIN  = 5;  // Pin GPIO 5 del ESP32

HX711 scale;

// Factor de calibración (ajustar según tu celda tipo botón)
float calibration_factor = 2280.0;

// Geometría de la Valva Sternal en pulgadas cuadradas (sq in)
const float AREA_VALVA_SQ_IN = 3.875;

// ========================================
// CONFIGURACIÓN BLE (ESP32)
// ========================================
#define SERVICE_UUID        "19B10000-E8F2-537E-4F6C-D104768A1214"
#define CHARACTERISTIC_UUID "19B10001-E8F2-537E-4F6C-D104768A1214"
```

```
37  BLECharacteristic *pCharacteristic;
38  bool deviceConnected = false;
39
40  class MyServerCallbacks: public BLEServerCallbacks {
41      void onConnect(BLEServer* pServer) {
42        deviceConnected = true;
43      }
44      void onDisconnect(BLEServer* pServer) {
45        deviceConnected = false;
46      }
47  };
48
49  unsigned long lastSendTime = 0;
50  const long interval = 500; // Actualizar cada 500 ms
51
52  void setup() {
53    Serial.begin(115200);
54
55    // Inicializar Pantalla OLED (Dirección I2C 0x3C)
56    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
57      Serial.println(F("Fallo al iniciar la pantalla OLED"));
58      while(1);
59    }
60    display.clearDisplay();
61    display.setTextSize(1);
62    display.setTextColor(SSD1306_WHITE);
63    display.setCursor(0,0);
64    display.print("Iniciando Pectus...");
65    display.display();
66    delay(1000);
67
68    // Inicializar Celda de Carga
69    scale.begin(HX711_DOUT_PIN, HX711_SCK_PIN);
70    scale.set_scale(calibration_factor);
71    scale.tare(); // Poner a cero
73    // Inicializar BLE en ESP32
74    BLEDevice::init("PectusBrace_ESP32");
75    BLEServer *pServer = BLEDevice::createServer();
76    pServer->setCallbacks(new MyServerCallbacks());
77
78    BLEService *pService = pServer->createService(SERVICE_UUID);
79    pCharacteristic = pService->createCharacteristic(
80                        CHARACTERISTIC_UUID,
81                        BLECharacteristic::PROPERTY_READ   |
82                        BLECharacteristic::NOTIFY
83                      );
84    pService->start();
85
86    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
87    pAdvertising->addServiceUUID(SERVICE_UUID);
88    pAdvertising->setScanResponse(true);
89    pServer->startAdvertising();
90
91    Serial.println("ESP32 BLE Servidor listo, esperando conexion...");
92  }
93
94  void loop() {
95    unsigned long currentTime = millis();
96
97    if (currentTime - lastSendTime >= interval) {
98      lastSendTime = currentTime;
99
100     // 1. Leer fuerza en Libras (lbs)
101     float fuerza_lbs = scale.get_units(5);
102     if (fuerza_lbs < 0) fuerza_lbs = 0.0;
103
104     // 2. Calcular Presión en PSI
105     float presion_psi = fuerza_lbs / AREA_VALVA_SQ_IN;
```

```
    // 3. Mostrar datos en la Pantalla OLED
    display.clearDisplay();
    display.setCursor(0, 0);
    display.setTextSize(1);
    display.print("Corse Pectus Care");
    display.setCursor(0, 16);
    display.print("Fuerza: ");
    display.print(fuerza_lbs, 1);
    display.print(" lbs");

    display.setCursor(0, 32);
    display.setTextSize(2);
    display.print(presion_psi, 1);
    display.print(" PSI");

    display.setCursor(0, 52);
    display.setTextSize(1);
    if (deviceConnected) {
      display.print("Estado: Conectado");
    } else {
      display.print("Estado: Esperando...");
    }
    display.display();

    // 4. Enviar datos por Bluetooth si hay conexión
    if (deviceConnected) {
      char txString[8];
      dtostrf(presion_psi, 1, 2, txString);
      pCharacteristic->setValue(txString);
      pCharacteristic->notify();
    }

    // Monitor Serie
    Serial.print("Fuerza: "); Serial.print(fuerza_lbs); Serial.print(" lbs | ");
    Serial.print("Presion: "); Serial.print(presion_psi); Serial.println(" PSI");
  }
}
}
```